Ders 8 Genişletilmiş Notu: Süreçler Arası İletişim ve Saga ile Transaction Yönetimi

Ders 8 Genişletilmiş Notu: Süreçler Arası İletişim ve Saga ile Transaction Yönetimi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin biçimi
- 2. Bölüm 3: Etkileşim stilleri
- 3. API tanımı ve anlamsal sürümleme
- 4. Mesaj biçimleri: metin ve ikili
- 5. Senkron iletişim: uzak yordam çağrısı (RPI) kalıbı
- 6. REST
- 7. gRPC
- 8. Asenkron mesajlaşma kalıbı
- 9. Asenkron mesajlaşma ve erişilebilirlik
- 10. Hocanın Bölüm 3 değerlendirmesi: RPC, REST ve mesaj aracısı
- 11. Bölüm 4: Mikroservis mimarisinde transaction yönetimi
- 12. Dağıtık transaction ve iki aşamalı commit
- 13. Saga kalıbı ve telafi edici transaction'lar
- 14. Sagaların koordinasyonu: koreografi
- 15. Sagaların koordinasyonu: orkestrasyon
- 16. Yalıtım eksikliği ve anomaliler
- 17. Bir saganın yapısı: telafi edilebilir, pivot ve yeniden denenebilir transaction'lar
- 18. Karşı önlemler
- 19. Bölüm 4 özeti ve hocanın değerlendirmesi

1. Dersin biçimi

Bu derste de öğrenci sunumlarıyla Chris Richardson'ın "Microservices Patterns" kitabı işlenir. İki bölüm sunulur: Bölüm 3, mikroservis mimarisinde süreçler arası iletişim (interprocess communication, IPC); Bölüm 4, saga ile transaction yönetimi. Hoca girişte üçüncü bölümün mikroservis mimarisindeki süreçler arası haberleşmeyi ve bunun türlerini incelediğini söyler (00:01:53).

İlk sunum önceden kaydedilmiş bir video olarak izletilir. Videodaki slaytlar ekranda okunur durumdadır; bu nedenle 2.-9. bölümlerdeki tanımlar esas olarak slaytlardan alınmış, sunumun Türkçe anlatımıyla tamamlanmıştır. İkinci sunum canlı yapılır. Hoca her sunumdan sonra konuyu kendi sözleriyle özetler.

2. Bölüm 3: Etkileşim stilleri

Bir mikroservis mimarisinde servisler ve istemciler arasındaki iletişim temel bir konudur. Sunum, istemci-servis etkileşimini iki boyutta sınıflandırarak başlar (ekrandan, 00:04:50):

- **Birinci boyut, bire bir mi bire çok mu:**

  - **Bire bir (one-to-one):** Her istemci isteği tam olarak bir servis tarafından işlenir.
  - **Bire çok (one-to-many):** Her istek birden çok servis tarafından işlenir.
- **İkinci boyut, senkron mu asenkron mu:**
  - **Senkron:** İstemci servisten zamanında bir yanıt bekler, hatta beklerken bloke olabilir.
  - **Asenkron:** İstemci bloke olmaz; yanıt varsa bile hemen gönderilmek zorunda değildir.

Slaytın son maddesi şunu hatırlatır: senkron istek/yanıt etkileşim stili, kullanılan IPC teknolojisinden büyük ölçüde bağımsızdır (orthogonal). Yani bir stil seçmek bir teknolojiyi seçmek anlamına gelmez.

Sunumda senkron ve asenkron seçiminin her işlem için ayrı yapıldığı da vurgulanır (00:19:07). Örnek olarak giriş (login) işlemi verilir: kullanıcı bilgilerini girer ve bu bilgilere yanıt gelmeden diğer işlemlere geçemez. Burada beklemek gerektiği için senkron bir yapı uygundur.

> **Ders sonrası not:** Sunumda bir yerde asenkron yapıda da yanıtın beklenmesi gerektiği söylenmiş (00:17:26), sunan öğrenci daha sonra araya girip senkron ile asenkronu karıştırdığını düzeltmiştir (00:21:25). Hoca da düzeltmeyi onaylar: beklemek gerekiyorsa senkron bir yapı kurgulamak daha doğrudur (00:21:36). Doğru tanım yukarıdaki slayttaki tanımdır: asenkronda istemci bloke olmaz.

3. API tanımı ve anlamsal sürümleme

3.1 Önce arayüz tanımı

İstemciler servisteki işlemleri çağırır, servis de bunlara yanıt verir. Bu yüzden bir API tasarlanırken işe bir arayüz tanımıyla (interface definition language, IDL) başlamak gerekir (00:05:50-00:06:13). Sunumdaki gerekçe şudur: ileride müşteri istekleri doğrultusunda değişiklik yapılsa bile, başlangıçta bir arayüz tanımı yapılıp onun üzerinden ilerlemek daha doğru ve sağlıklıdır.

3.2 API'yi değiştirmek neden zor?

Mikroservis tabanlı uygulamalarda API'yi değiştirmek zordur (00:06:37). Bir servisin API'si birçok farklı yerde kullanılıyor olabilir; bir yer için düşünülen değişiklik birçok istemciyi etkiler.

3.3 Anlamsal sürümleme (SemVer)

Slayttaki maddeler (ekrandan, 00:07:25):

- Anlamsal sürümleme ilk olarak yazılım paketlerinin sürümlenmesi için düşünülmüştür, ama dağıtık bir sistemdeki API'leri sürümlemek için de kullanılabilir.
- Semantic Versioning belirtimi, sürüm numarasının üç parçadan oluşmasını ister: `MAJOR.MINOR.PATCH`. Her parça şöyle artırılır:
  - **MAJOR:** API'ye uyumsuz (incompatible) bir değişiklik yapıldığında.
  - **MINOR:** API'ye geriye uyumlu (backward-compatible) iyileştirmeler yapıldığında.
  - **PATCH:** Geriye uyumlu bir hata düzeltmesi (bug fix) yapıldığında.

Sunumda bunun herkesin tanıdığı bir konu olduğu, 1.9.2 gibi sürümlerin sık görüldüğü söylenir (00:07:13). Her değişiklikte ilgili parça artırılır.

4. Mesaj biçimleri: metin ve ikili

Mikroservis mimarisinde her noktada bir iletişim vardır; bu iletişimin hangi biçimdeki verilerle sağlanacağı önemlidir (00:09:26). Mesaj biçimleri metin tabanlı ve ikili (binary) olarak iki gruba ayrılır.

4.1 Metin tabanlı biçimler

Slayttaki maddeler (ekrandan, 00:10:00):

- JSON ve XML gibi metin tabanlı biçimler kendini tanımlar (self-describing) ve insan tarafından okunabilir.
- Mesaj şemasındaki birçok değişiklik kolayca geriye uyumlu yapılabilir.
- Sakıncası: mesajlar uzun olma eğilimindedir, özellikle XML. Bir başka sakınca da metni ayrıştırmanın (parsing) getirdiği yüktür, özellikle mesajlar büyükse. Verim ve performans önemliyse ikili bir biçim düşünülmelidir.

Sunumda XML'deki etiketleme (tag) yapısının insanların mesajı kolayca anlamasını sağladığı, JSON'un da anahtar-değer (key-value) çiftlerinden oluştuğu söylenir (00:10:00-00:10:12). XML etiketleri yüzünden JSON'a göre daha büyük mesajlar üretir ve bu performans sorununa yol açabilir (00:10:28).

4.2 İkili biçimler

Slayttaki maddeler (ekrandan, 00:11:00):

- Popüler biçimler Protocol Buffers ve Avro'dur. İkisi de mesajların yapısını tanımlamak için tipli bir IDL sunar.
- Aralarındaki bir fark: Protocol Buffers etiketli alanlar (tagged fields) kullanır; Avro tüketicisi ise mesajları yorumlayabilmek için şemayı bilmek zorundadır.

Sunumun sonucu: mikroservisin neye ihtiyaç duyduğu göz ardı edilmeden, gerekene göre bir biçim seçilmelidir (00:10:56).

5. Senkron iletişim: uzak yordam çağrısı (RPI) kalıbı

Slayttaki tanım (ekrandan, 00:11:55): uzak yordam çağrısı (remote procedure invocation) tabanlı bir IPC mekanizmasında istemci servise bir istek yollar; servis isteği işler ve bir yanıt geri gönderir. Slayttaki şekil metne dökülürse:

```
İstemci (altıgen):  [İş mantığı] --çağırır--> (Proxy arayüzü) = [RPI proxy]
                    [RPI proxy] ----- Request -----> [RPI server]
Servis (altıgen):   [RPI server] --> (Servis arayüzü) --> [İş mantığı]
                    [RPI proxy] <----- Reply ------- [RPI server]
```

İstemcinin iş mantığı bir proxy arayüzünü çağırır. Bu arayüzü gerçekleyen RPI proxy, isteği servis tarafındaki RPI sunucusuna gönderir. RPI sunucusu servis arayüzü üzerinden servisin iş mantığını çağırır, bir sonuç üretir ve bu sonuç yanıt olarak istemciye döner (00:11:47-00:12:10). İstemci ve servis, önceki derste görülen altıgen (hexagonal) mimariyle çizilmiştir.

6. REST

6.1 REST neden önemli?

Sunumda REST mimarisinin, istemci ile sunucu arasındaki iletişimi daha ölçeklenebilir ve kapsüllenmiş bir yapıyla sağladığı, özellikle HTTP protokolünün kullanıldığı yerlerde geniş bir kullanım alanı olduğu söylenir (00:12:21-00:12:31).

6.2 REST olgunluk modeli

Slayttaki maddeler (ekrandan, 00:12:55): Leonard Richardson, REST için çok yararlı bir olgunluk modeli tanımlar. Model şu seviyelerden oluşur:

- **Seviye 0:** Seviye 0 bir servisin istemcileri, servisi onun tek URL uç noktasına (endpoint) HTTP POST istekleri yaparak çağırır.
- **Seviye 1:** Bir kaynak üzerinde işlem yapmak için istemci, yapılacak işlemi ve parametreleri belirten bir POST isteği yapar.
- **Seviye 2:** Servislerin, GET istekleri için önbellekleme (caching) gibi web altyapısını kullanmasını sağlar.
- **Seviye 3:** Bir GET isteğiyle dönen kaynağın gösterimi (representation), o kaynak üzerinde işlem yapmak için bağlantılar (links) içerir.

Ek bilgi (derste anlatılmadı): Seviye 3 literatürde büyük olasılıkla HATEOAS (Hypertext As The Engine Of Application State) adıyla bilinir. Seviye 2'de servisler kaynaklar üzerindeki işlemleri HTTP fiilleriyle (GET okuma, POST oluşturma, PUT güncelleme gibi) ayırır; GET önbelleklemesi bu ayrımın sonucudur.

6.3 REST'te IDL

Sunumda REST'in kendi içinde bir IDL barındırmadığı, bu yönüyle CORBA ve SOAP'tan ayrıldığı söylenir (00:13:49). Bu yüzden API tasarımında POST ve GET gibi yöntemlere ve bunların işlemlerine ihtiyaç duyulur; parametreler URL üzerinden geçirilir. Sunumdaki değerlendirmeye göre bu, hem geliştirmede hem anlaşılmada fazladan zaman harcatabilir (00:14:05-00:14:27).

6.4 Yararları ve sakıncaları

Slayttaki maddeler (ekrandan, 00:14:45):

- REST'i kullanmanın yararları:
  - Basit ve tanıdıktır.
  - HTTP API'si bir tarayıcıdan, Postman ile test edilebilir.
  - İstek/yanıt stili iletişimi doğrudan destekler.
- REST'i kullanmanın sakıncaları:
  - Yalnız istek/yanıt stilindeki iletişimi destekler.
  - Tek bir istekte birden çok kaynağı getirmek zordur.
  - Birden çok güncelleme işlemini HTTP fiillerine eşlemek bazen zordur.

Sunumda yalnız istek/yanıt desteklemenin sonucu bir sayfa örneğiyle anlatılır (00:15:00-00:15:25): servis tarafında bir değişiklik olduğunda, sayfa yenilenmedikçe ya da arka planda bir istek yapılmadıkça değişiklik sayfaya gelmez. REST abone olma (subscribe) mekanizmasına uygun değildir. Karmaşık bir yapıda çoklu veri çekmek de URL'e verilen parametreler yüzünden işi zorlaştırır ve hata yönetimini güçleştirir (00:15:33-00:15:42).

7. gRPC

Sunumda gRPC'ye geçmeden önce mesaj protokollerinin neden önemli olduğu bağlanır: mikroservisler kendi başına birimlerdir ve bir araya gelebilmeleri için iletişim kurmaları gerekir (00:16:01). Slayttaki maddeler (ekrandan, 00:16:30):

- gRPC ikili mesaj tabanlı bir protokoldür; servis tasarımında API-first (önce API) yaklaşımını zorunlu kılar.
- gRPC mesaj biçimi olarak Protocol Buffers kullanır. Protocol Buffers verimli, kompakt, ikili bir biçimdir. gRPC, API'lerin geriye uyumlu kalarak evrilmesini sağlar.
- gRPC'nin yararları:
  - Zengin bir güncelleme işlemleri kümesine sahip bir API tasarlamak kolaydır.
  - Özellikle büyük mesajlar değiş tokuş edilirken verimli, kompakt bir IPC mekanizmasıdır.
  - Çift yönlü akış (bidirectional streaming) hem RPI hem mesajlaşma stilindeki iletişimi mümkün kılar.

Sunumda kompaktlığın hem iletim süresi hem barındırma maliyeti açısından kolaylık sağladığı, performans kaygısı varsa ikili biçimlerin devreye girdiği vurgulanır (00:16:47-00:17:06).

8. Asenkron mesajlaşma kalıbı

8.1 Mesajın yapısı

Slayttaki maddeler (ekrandan, 00:17:50):

- Bir mesaj başlık (header) ve mesaj gövdesinden (body) oluşur.
- Başlık, gönderilen veriyi tanımlayan meta veriler olan ad-değer (name-value) çiftlerinin bir topluluğudur. Gönderenin verdiği ad-değer çiftlerine ek olarak başlık, gönderen ya da mesajlaşma altyapısı tarafından üretilen benzersiz bir mesaj kimliği (message id) ve

isteğe bağlı bir dönüş adresi (return address; yanıtın yazılması gereken mesaj kanalı) içerir.
- Mesaj gövdesi, metin ya da ikili biçimde gönderilen veridir.
- Birkaç farklı mesaj türü vardır:
  - **Belge (document):** Yalnız veri içeren genel bir mesaj.
  - **Komut (command):** Bir RPC isteğinin eşdeğeri olan mesaj.
  - **Olay (event):** Gönderende kayda değer bir şeyin olduğunu bildiren mesaj.

Sunumda başlığın neden önemli olduğu bir resim örneğiyle anlatılır (00:18:13-00:18:51): PNG ve JPEG formatında birer resim gönderildiğinde ikisi de sonuçta resimdir; ama başlıklarına bakıldığında çok farklı oldukları görülür ve alıcının resmi nasıl işleyeceği (render edeceği) buna göre değişir. Bu noktada başlık ve gövde önemli rol oynar.

8.2 Mesajlaşma tabanlı bir servis API'si belirtmek

Slayttaki maddeler (ekrandan, 00:20:00):

- Mesaj kanallarının adları, her kanal üzerinden değiş tokuş edilen mesaj türleri ve bunların biçimleri belirtilir. Mesajların biçimi JSON, XML ya da Protobuf gibi bir standartla tanımlanmalıdır.
- Bir servisin asenkron API'si, istemcilerin çağırdığı işlemlerden ve servisin yayımladığı olaylardan oluşur.

Sunumdaki açıklamaya göre mesaj aktarımında genellikle ayrıştırmaya gitmemek, olabildiğince aynı biçimde veri aktarmak hedeflenir (00:20:05). Ama bu her zaman mümkün değildir: örneğin JSON, XML ve XML olarak gelen üç veri ortak bir biçimde toplanacaksa API bazında özelleştirmeler yapılabilir. Mesajların gönderilme sırası da hem alıcı hem gönderen için önemlidir (00:20:33).

8.3 Mesaj aracıları, kütüphaneler ve çerçeveler

Mesajları iletmek için mesaj aracılarına (message broker) ihtiyaç duyulabilir (00:23:08). Mesaj aracıları genelde düşük seviyede çalışır ve çok kod gerektirebilir; bu yüzden üzerlerinde kütüphaneler ve çerçeveler (framework) kullanılır (00:23:14-00:23:24).

9. Asenkron mesajlaşma ve erişilebilirlik

Slayttaki maddeler (ekrandan, 00:23:35):

- REST son derece popüler bir IPC mekanizmasıdır ve servisler arası iletişim için kullanmak cazip gelebilir. Ancak REST'in sorunu senkron bir protokol olmasıdır: HTTP istemcisi servisin yanıt göndermesini beklemek zorundadır. Servisler senkron bir protokolle haberleştiği her durumda uygulamanın erişilebilirliği (availability) düşer.
- Bu sorun REST tabanlı iletişime özgü değildir. Bir servis istemcisine ancak başka bir servisten yanıt aldıktan sonra yanıt verebiliyorsa erişilebilirlik düşer. Servisler asenkron mesajlaşma üzerinden istek/yanıt stiliyle haberleşse bile bu sorun vardır.

Sunumdaki anlatım (00:24:03-00:24:25): istemci senkron bir istekte yanıt bekler; yanıt süresi uzunsa istemci sürekli bekler ve gerçek hayatta bu kullanışsız bir yapıdır. Kitabın yaklaşımı senkron yapıların olabildiğince ortadan kaldırılmasıdır; böylece performansın artması beklenir.

> **Ders sonrası not:** Sunumun sonunda HTTP üzerinde REST ile asenkron iletişim de sağlanabildiği ve RESTful API'nin "daha asenkron temelli" bir iletişim sağladığı söylenir (00:23:32-00:24:46). Bu, aynı sunumdaki slaytla çelişir: slayt REST'in senkron bir protokol olduğunu açıkça yazar (ekrandan, 00:23:35). Kitapta senkron bağımlılığı azaltmak için önerilen yol büyük olasılıkla REST değil, mesaj aracısı üzerinden asenkron mesajlaşmadır.

10. Hocanın Bölüm 3 değerlendirmesi: RPC, REST ve mesaj aracısı

Hoca sunumdan sonra bölümü kendi sözleriyle özetler. Elimizde servisler vardır, bunlar mikroservisler olabilir ve birlikte çalışarak iş yapabilmeleri için birbirleriyle haberleşmeleri, "konuşmaları" gerekir (00:25:40).

10.1 RPC: telefon hattı gibi sıkı bağlılık

En eski ve en temel teknik RPC'dir; Türkçesi uzak yordam çağrısıdır (Remote Procedure Call) (00:25:56). RPC ile gevşek değil, sıkı bağlı (tightly coupled) bir yapı elde edilir: iki servis birbiriyle ciddi bir iletişim kanalı açarak konuşur. Hoca bunu telefon hattına benzetir: hat bu konuşmaya adanmıştır (dedicated) ve haberleşme bitene kadar açık kalır. Bu, iki tarafın da meşgul olması demektir; hem alan hem veren birbirini beklemek zorundadır (00:26:24).

Eskiden RMI (Remote Method Invocation) da vardı; o da süreçler arası iletişim için tasarlanmış bir çerçeve ya da kütüphaneydi (00:26:52). Haberleşmenin teknik ayrıntılarıyla RPC kütüphanesi ilgilenir; servis kendi iş mantığında, yani altıgen mimarinin merkezinde kalır ve yalnız bir arayüze bağlanarak veri alıp verir (00:27:08). Ama birbirini bekleme, telefon hattı gibi meşgul tutma iyi bir şey değildir: servisler kilitlenir, yeni istek alınamaz, hizmet verilemez (00:27:34).

10.2 Asenkron yaklaşım ve REST

Hoca bunun karşısına "isteyelim, yollayalım, sonra çalışsın, sonucu bana döndürsün; ben de o sırada onu beklemeyeyim" şeklindeki asenkron haberleşme yaklaşımlarını koyar ve bu bağlamda REST'i (Representational State Transfer) anar (00:27:45-00:27:59). Her yaklaşımın artıları ve eksileri vardır; hangisinin kullanılacağı, senkron mu asenkron mu olacağı ve hangi protokolün seçileceği mimari tasarlanırken ölçülüp biçilerek belirlenir (00:28:07). Genel uygulamalar ve kütüphaneler çoğunlukla RESTful servisler şeklinde tasarlanmaya çalışılır (00:28:34).

> **Ders sonrası not:** Hoca bu kısımda REST'i asenkron haberleşme yaklaşımının örneği gibi anlatır (00:27:45). Sunumdaki slayta göre REST senkron bir protokoldür (ekrandan, 00:23:35). Hocanın asıl vurgusu (birbirini bekleyip kilitlenmeyen servisler) doğrudur; ancak bu özellik büyük olasılıkla REST'ten değil, asenkron mesajlaşmadan ve olay yayımlamadan gelir.

REST'te bir şey değiştiğinde haberdar olma (yenileme) sorunu için hoca olay yayımlama (event publish) mekanizmalarını hatırlatır: bir mesaj yayımlanır, o mesajla ilgilenenler onu dinler ve ona göre işlem yapar; böylece güncelleme yayılmış olur (00:28:48-00:28:57).

10.3 Mesaj aracısı: trafik polisi

Servisler bölündü, birbirini beklemiyor, asenkron çalışabiliyor: birbirine mesaj yollayıp isterse bekleyip, istemezse o arada başka işler yapıp cevabı aldığında işine devam edebiliyor (00:29:16). Ama servisler çok örnekli (multi-instance) çalışır ki aynı hizmet aynı anda birden fazla istemciye sunulabilsin (00:29:44). Bu durumda mesajlaşmanın yönetimi, orkestrasyonu gerekir: hangi servis hangi mesajı alacak, hangisini aldı, hangisini işleyecek, bu cevabı kim bekliyordu, kime yollanacak? Mesajlar bir kuyruk mekanizması üzerinde yönetilmeli ve buradan aktarılmalıdır. Buna message broker ya da message queue denir (00:30:00).

Rabbit (RabbitMQ) gibi hazır çözümler en çok kullanılanlardandır. Mesaj kuyrukları gelen istekleri sıraya dizer ve hangi servisin hangi istekle ilgileneceğini, hangi isteğin hangi tarafa kanalize edileceğini belirler. Hoca bunu trafiğin aktarılmasını ve yönetilmesini sağlayan bir trafik polisine benzetir (00:30:25-00:30:42).

> **Hocanın vurgusu:** Tek bir doğru yoktur. Mesajlaşma sistemi ve servis haberleşmesi uygulamanın doğasına göre seçilir ve tasarlanır. Bazı uygulamalarda, hatta bazı uygulamaların bazı parçalarında senkron gerekirken bazılarında asenkron gerekecektir. Çoğunlukla asenkron yapmaya gayret ederseniz birbirini beklemeyen, kilitlenmeyen ve genişlemeye yatkın sistemleri daha kolay kurarsınız; ama bazen mutlaka senkronda kalmanız gerekebilir (00:30:51-00:31:41).

11. Bölüm 4: Mikroservis mimarisinde transaction yönetimi

Hoca ikinci sunumu şu sorularla açar: transaction nedir, transaction yönetimi nasıl yapılır, mikroservis gibi dağıtık bir ortamda bütün işlemler nasıl gerçekleştirilir (00:32:00)?

> **Ders sonrası not:** Hoca transaction için "toplu işlem, bulk operation" karşılıklarını anar (00:32:00). Transaction, büyük olasılıkla daha doğru bir karşılıkla "bölünmez işlem" ya da "hareket"tir: bir bütün olarak ya tamamen gerçekleşen ya da hiç gerçekleşmeyen işlem dizisi. Toplu işlem (bulk operation) çok sayıda kaydı tek seferde işlemek anlamına gelen ayrı bir kavramdır.

11.1 Sunumun planı

Sunum üç başlıktan oluşur (ekrandan, 00:33:40):

1. Mikroservis mimarisinde transaction yönetimi.
2. Sagaların koordinasyonu (ve türleri).
3. Yalıtım eksikliğiyle başa çıkmak (sagaların en büyük sorunu).

11.2 Transaction ve ACID

Sunumdaki tanım: veritabanında bir değişikliğe yol açan her türlü işleme transaction denir (00:33:56). Bu işlemlerin bazıları başka işlemleri tetikler. Mikroservis mimarisinde bu, birden çok servisin art arda çalışması demektir ve art arda çalışan bu transaction'ların yönetilmesi gerekir.

Örnek bir e-ticaret sitesidir (00:34:25): önce ödeme işlemi yapılır, sonra ürün stoktan düşülür. Sıralı gerçekleşecek bir dizi transaction vardır. Ödeme gerçekleştikten hemen sonra bir hata olursa ne olacak? İşlemler geri mi alınacak, devam mı edilecek? Stoktan düşme işlemi yapıldıysa süreç başa mı döndürülecek? Bölüm bu soruyla ilgilenir.

Slayttaki soru ve cevap (ekrandan, 00:35:10): işlemler için transaction'lar birden çok servisteki veriyi güncelliyorsa ne yapacağız? Farklı servisler arasında çalışan bir transaction yönetim mekanizmasına ihtiyaç vardır. Slaytın sağında ACID özellikleri sıralanır:

```
A  ->  Atomicity     (bölünmezlik)
C  ->  Consistency   (tutarlılık)
I  ->  Isolation ?   (yalıtım)
D  ->  Durability    (kalıcılık)
```

Isolation'ın yanındaki soru işareti, sunumun sonunda görülecek olan sorunun habercisidir: ACID özellikleri korunmaya çalışılır ama tamamıyla korunamaz (00:35:00).

11.3 Monolit ve mikroservis farkı

Monolitik mimaride transaction yönetimi çok daha kolaydır (00:35:23). Çoğu çerçeve kendi içinde transaction API'leri içerir ve uygulamanın tek bir veritabanı olduğu için bütün transaction'lar onun üzerinden çalışır. Mikroservis mimarisinde ise bir transaction birçok servisi etkiler ve bu servislerin her birinin kendi veritabanı vardır (00:35:40). Geleneksel yöntem burada kullanılamaz; ilk akla gelen çözüm dağıtık transaction'lardır.

12. Dağıtık transaction ve iki aşamalı commit

Dağıtık transaction protokolü ACID ilkelerini korumayı sağlar. Bunun yaygın biçimi iki aşamalı commit'tir (Two Phase Commit, 2PC) (ekrandan, 00:36:20). İki aşamayı yöneten bir koordinatör vardır:

- **Hazırlık (prepare) aşaması:** Koordinatör katılımcılara transaction'ı tamamlamaya hazır olup olmadıklarını sorar.
- **Commit aşaması:** İşlemler kalıcı olarak kaydedilir.

E-ticaret örneğinde (00:36:29): akış başlar, transaction'lar tamamlanır. Koordinatör transaction'lara başarılı olup olmadıklarını sorar. Hem ödeme hem stoktan düşme "tamam, sorun yok"

yanıtını verirse commit aşamasına geçilir ve işlemler diske yazılır. Koordinatör katılımcılardan birinden başka bir yanıt alırsa bütün transaction'lar iptal edilir. Atomicity böyle sağlanır.

Slayttaki sorunlar (ekrandan, 00:36:20):

- Modern teknolojiler dağıtık transaction'ları desteklemez (örnek: MongoDB gibi NoSQL veritabanları).
- Bir transaction'ın tamamlanması için katılan bütün servislerin erişilebilir olması gerekir; bu da erişilebilirliği düşürür.

Bu sorunlar sagaların kullanılmasına yol açar (00:36:56).

13. Saga kalıbı ve telafi edici transaction'lar

13.1 Saga nedir?

Slayttaki tanım (ekrandan, 00:37:50):

- **Saga:** Asenkron mesajlaşmayla koordine edilen bir yerel transaction dizisi kullanarak servisler arasında veri tutarlılığını korur.
- Bir yerel transaction'ın tamamlanması, bir sonraki yerel transaction'ın çalışmasını tetikler.

Sunumdaki anlatımla (00:37:21): saga, her transaction'ın farklı ve bağımsız bir servis üzerinde yerel olarak çalıştığı ve verisini yine o serviste güncellediği bir transaction dizisidir. İlk transaction tetiklenir, onun tamamlanması başka bir transaction'ı tetikler.

13.2 Restoran siparişi örneği

Slayttaki şekil (ekrandan, 00:37:50) dört servise yayılan altı yerel transaction gösterir:

```
Order Service       Consumer Service    Kitchen Service      Accounting Service
Txn 1: Create order
                    Txn 2: Verify consumer
                                        Txn 3: Create ticket
                                                             Txn 4: Authorize card
                                        Txn 5: Approve ticket
Txn 6: Approve order
```

Sunumdaki anlatım (00:37:44-00:38:11): Order Service siparişi oluşturur. Consumer Service müşterinin sipariş verebileceğini doğrular. Kitchen Service sipariş ayrıntılarını alır ve mutfak fişini (ticket) oluşturur. Accounting Service kredi kartı bilgisini onaylar. Sonra Kitchen Service'e dönülür ve fişin durumu "onaylandı" yapılır. En son Order Service'e dönülür ve siparişin durumu "onaylandı" yapılır.

> **Ders sonrası not:** Sunumda Kitchen Service'in oluşturduğu "ticket" için "fatura" denir (00:37:44). Kitaptaki örnekte ticket, mutfağın hazırlayacağı siparişi temsil eden mutfak fişidir; ödeme ve fatura işleri Accounting Service'tedir.

13.3 Telafi edici transaction'lar

ACID'in bir yanı geri almadır (rollback): hata durumunda işlemin tamamen iptal edilmesi (00:38:18). Sagalarda bu yapılamaz, çünkü her yerel transaction kendi verisinde değişiklik yapıp commit etmiştir (00:38:26). Bunun yerine telafi edici transaction'lar (compensating transactions) kullanılır.

Slayttaki şekil (ekrandan, 00:38:50):

```
Saga:
  | T1 | ... | Tn | T(n+1) FAILS | Cn | ... | C1 |
  \_____________/                  \_____________/
  T1...Tn'nin yaptığı               Telafi edici transaction'lar
  değişiklikler commit edildi.      T1...Tn'nin değişikliklerini geri alır.
```

T1'den Tn'ye kadar transaction'lar başarıyla gerçekleşmiştir. Sonraki adımda, örneğin kredi kartı onayında, hata olursa o ana kadar yapılmış işlemler tek tek geri alınır: her transaction'a karşılık gelen telafi edici transaction çalıştırılır ve işlem sondan başa doğru geri döndürülür (00:38:41-00:39:08).

> **Ders sonrası not:** Sunumda hatanın "Tn-1" noktasında olduğu söylenir (00:38:41). Slayttaki şekle göre başarısız olan transaction T(n+1)'dir; telafi Cn'den C1'e doğru ilerler.

14. Sagaların koordinasyonu: koreografi

Sagayı uygulamanın iki yolu vardır: koreografi (choreography) ve orkestrasyon (orchestration) (00:39:20).

14.1 Koreografi nasıl çalışır?

Slayttaki maddeler (ekrandan, 00:40:00):

- Merkezî bir koordinatör yoktur.
- Katılımcılar birbirinin olaylarına yanıt verir.
- Transaction tamamlanana kadar olay alışverişi sürer.

İlk servis işini tamamlayınca bir olay gönderir; bu olayı dinleyen servisler tetiklenir ve kendi yerel transaction'larını çalıştırır. Her servis, öncekinin işini bitirdiğini ve sıranın kendisine geldiğini haber vermesini bekler. Son servis çalıştıktan sonra artık olay fırlatılmaz ve süreç sona erer (00:39:33-00:39:51).

Slayttaki şekil metne dökülürse (ekrandan, 00:40:00). Ortada bir mesaj aracısı ve dört olay kanalı vardır (Order events, Consumer events, Ticket events, Credit card events):

```
1. Order Service: createOrder()           -> "Order created" yayımlar (Order events)
2. Consumer Service: verifyConsumerDetails() -> "Consumer verified" yayımlar
3. Kitchen Service: createTicket()        -> "Ticket created" yayımlar
4. Accounting Service: createPendingAuthorization()
5. Accounting Service: authorizeCard()    -> "Credit card authorized" yayımlar
6. Kitchen Service: approveTicket()
7. Order Service: approveOrder()
```

Sunumdaki anlatım (00:39:58-00:40:32): Order Service siparişi oluşturur ve "Order Created" olayını gönderir. Consumer Service bu olayı görür, siparişin verilebileceğini onaylar ve "Consumer Verified" olayını gönderir. Kitchen Service "Order Created" olayına yanıt verir, siparişi alır, fişi oluşturur ve "Ticket Created" olayını gönderir. Accounting Service "Order Created" olayına yanıt verir ve kredi kartı onayını bekleyen bir durum oluşturur. Her transaction bir öncekinin olayına yanıt verip bir sonrakine haber verir.

14.2 Yararları ve sakıncaları

Slayttaki maddeler (ekrandan, 00:41:15):

- **Yararları:**
  - **Basitlik:** Servisler bir şey oluşturduklarında ya da her güncellemede olay yayımlar. Sunumdaki ifadeyle teorik mantığı çok basittir.
  - **Gevşek bağlılık (loose coupling):** Katılımcılar birbirinden bağımsızdır. Olaylarla haberleştikleri için birbirleri hakkında çok bilgileri yoktur; yalnız olaydan aldıkları bilgiyi bilirler (00:41:04).
- **Sakıncaları:**
  - **Anlamak daha zordur:** Saganın gerçeklemesi olaylara dağılmıştır; bir geliştiricinin yapıyı okuyup tümüyle anlaması zorlaşır.
  - **Servisler arasında döngüsel bağımlılıklar:** Her saga adımı bir başkasının olayına bağlanır ve döngüler oluşabilir. A servisi B'ye haber verir ama B de A'ya geri dönecek bir haber vermek zorunda kalabilir. Bu döngünün herhangi bir noktasında hata olursa ne olacağı bir risktir (00:41:23).

- **Sıkı bağlılık riski (risk of tight coupling):** Her katılımcı kendini etkileyen olaylara bağlıdır; bir yerdeki değişiklik her şeyi etkileyebilir (00:41:43).

15. Sagaların koordinasyonu: orkestrasyon

15.1 Orkestrasyon nasıl çalışır?

Slayttaki maddeler (ekrandan, 00:42:50):

- Saga katılımcılarına ne yapacaklarını söyleyen bir orkestratör sınıfı tanımlanır.
- İletişim için komut/asenkron yanıt (command/async reply) stilinde etkileşim kullanılır.

Sunumdaki benzetmeyle orkestratör bir orkestra şefi gibidir: "sen bunu yap, sen şunu yap" diye herkese ne yapacağını söyler (00:42:01). Burada olay yoktur. Orkestratör bir adımı gerçekleştirmek için bir komut mesajı gönderir; görevli katılımcı işlemini yapar ve orkestratöre bir yanıt mesajı gönderir; orkestratör de bir sonraki adımı başlatır (00:42:26). Sunumda bu etkileşimin, ilk sunumda anlatılan asenkron istek/yanıt etkileşimi olduğu belirtilir.

Slayttaki şekil metne dökülürse (ekrandan, 00:42:50). Create Order Saga orkestratörü Order Service'in içindedir; komutlar her servisin istek kanalına (request channel), yanıtlar "Create order saga reply channel" kanalına gider:

Adım	Mesaj türü	Gönderen -> Alıcı	Mesaj
1	komut	Orkestratör -> Consumer Service	Verify consumer
2	yanıt	Consumer Service -> Orkestratör	Consumer verified
3	komut	Orkestratör -> Kitchen Service	Create ticket
4	yanıt	Kitchen Service -> Orkestratör	Ticket created
5	komut	Orkestratör -> Accounting Service	Authorize card
6	yanıt	Accounting Service -> Orkestratör	Card authorized
7	komut	Orkestratör -> Kitchen Service	Approve restaurant order
8	komut	Orkestratör -> Order Service	Approve order

Sunumda 8. adımın önemli olduğu vurgulanır (00:43:20): orkestratör Order Service içinde gerçeklendiği hâlde "Approve order" isteğini Order Service'in istek kanalına gönderir ve istek kendisine döner. Orkestratör bu güncellemeyi doğrudan kendi içinde de yapabilirdi; ama sistemdeki mesaj ve yanıt ilkesini bozmamak, kendi içinde tutarlı olmak için böyle yapar (00:43:34).

15.2 Yararları ve sakıncaları

Sunumda sayılan yararlar (00:43:48-00:44:33):

- **Daha basit bağımlılıklar:** Koreografideki gibi döngüler oluşmaz. Orkestratör saga katılımcılarını etkiler ama katılımcılar onu etkilemez; iletişim tek yönlüdür, döngüsel değildir.
- **Daha az bağlılık:** Her servis orkestratörün çağırdığı bir API'yi gerçekler; olaylar hakkında bilgi sahibi olmasına gerek yoktur.
- **Sorumlulukların ayrılması ve iş mantığının basitleşmesi:** Bütün koordinasyon orkestratörde toplanır. Katılımcı servisler diğer sagalar hakkında pek bir şey bilmez; yalnız gelen komuta ve verdikleri yanıta bakar.

Sakıncası: işin yükünü tamamen orkestratöre yüklemek ve onu merkezileştirmek bir risk olabilir (00:44:39).

> **Ders sonrası not:** Sunumda bu yüzden orkestratörün basit sagalarda kullanılmasının önerildiği söylenir (00:44:49). Kitapta öneri büyük olasılıkla tersidir: en basit sagalar dışında orkestrasyon önerilir, merkezileşme riskine karşı da orkestratör yalnız adımların sıralamasından sorumlu tutulur ve iş mantığı katılımcı servislerde bırakılır.

16. Yalıtım eksikliği ve anomaliler

16.1 Yalıtım nedir?

Yalıtım (isolation), aynı anda çalışan birden çok transaction'ın, sırayla çalışmış gibi aynı sonucu vermesini sağlar (00:45:05). Veritabanında her transaction'ın veriye özel erişimi varmış gibi

bir yanılsama yaratır; böylece eş zamanlı işlemler karışmadan işlenir (00:45:22).

16.2 Sagalarda neden yalıtım yok?

Sagaların sorunu yalıtım özelliğinin olmamasıdır (00:45:33). Bir saganın yerel transaction'ı tamamlandığında diğer sagalar bu değişikliği hemen görebilir. Bu iki soruna yol açar:

- Diğer sagalar, bir saga hâlâ çalışırken onun kullandığı veriyi değiştirebilir.
- Diğer sagalar, saga işini tamamlamadan onun yaptığı değişikliği okuyup buna göre hareket edebilir; oysa o saga işlemi tamamlayamamış ya da onaylamamış olabilir. Bu da tutarsız veriye yol açar.

Bu yüzden sagalar ACID yerine ACD modeli olarak değerlendirilir; yalıtım yoktur (00:45:54). Yalıtım eksikliği anomali denen sorunlara yol açar: eş zamanlı işlemler, sırayla çalışsalardı verecekleri sonuçtan farklı sonuç verir (00:46:19). Bunlarla mücadele etmek için karşı önlemler (countermeasures) kullanılır.

16.3 Anomaliler

Slayttaki iki anomali (ekrandan, 00:46:55):

- **Kayıp güncellemeler (lost updates):** Bir saga, başka bir saganın yaptığı güncellemenin üzerine yazar.
- **Kirli okumalar (dirty reads):** Bir saga, başka bir saganın güncellemekte olduğu veriyi okur.

Sunumda üçüncü bir tür olarak bulanık/tekrarlanamaz okumalar (fuzzy/nonrepeatable reads) da anılır (00:46:48).

> **Ders sonrası not:** Sunumda kitabın üçüncü türden bahsetmediği söylenir (00:46:48). Kitap büyük olasılıkla üç anomaliyi de tanımlar (bulanık/tekrarlanamaz okuma: bir saganın iki adımının aynı veriyi okuyup farklı sonuç alması, çünkü arada başka bir saga güncelleme yapmıştır), ama en yaygın olanların kayıp güncelleme ve kirli okuma olduğunu söyler.

Kayıp güncelleme örneği (00:47:13-00:47:27): Create Order Saga bir sipariş oluşturur. Bu saga çalışmaya devam ederken Cancel Order Saga siparişi iptal eder. Create Order Saga işine devam eder ve siparişi yine de onaylar. İptal güncellemesi gözden kaçmış, kaybolmuştur.

Kirli okumada ise bir saga, değişikliğe uğrayan bir veriyi işlemin ortasında okur; işlemin son hâlini göremediği için yanlış okuma yapmış olur (00:47:35).

17. Bir saganın yapısı: telafi edilebilir, pivot ve yeniden denenebilir transaction'lar

Karşı önlemlere geçmeden önce üç transaction türü tanımlanır. Slayttaki maddeler (ekrandan, 00:48:20):

- **Telafi edilebilir transaction'lar (compensatable):** Bir telafi edici transaction ile potansiyel olarak geri alınabilen transaction'lar. Geri almayı desteklemelidirler.
- **Pivot transaction:** Sagadaki devam/dur (go/no-go) noktası. Pivot transaction commit ederse saga tamamlanana kadar çalışır. Pivot transaction, ne telafi edilebilir ne yeniden denenebilir olan bir transaction olabilir.
- **Yeniden denenebilir transaction'lar (retriable):** Pivot transaction'ı izleyen ve başarılı olması garanti olan transaction'lar.

Slayttaki tablo (ekrandan, 00:48:20):

Adım	Servis	Transaction	Telafi edici transaction
1	Order Service	createOrder()	rejectOrder()
2	Consumer Service	verifyConsumerDetails()	-
3	Kitchen Service	createTicket()	rejectTicket()
4	Accounting Service	authorizeCreditCard()	-
5	Restaurant Order Service	approveRestaurantOrder()	-

```
6     Order Service             approveOrder()              -

1-3: telafi edilebilir     4: pivot     5-6: yeniden denenebilir
```

Sunumdaki açıklama (00:48:37-00:49:34): Order Service, Consumer Service ve Kitchen Service adımları telafi edilebilir transaction'lardır; bu süre içinde bunlardan birinde hata olursa süreç telafi edilebilir: geri gidilir, iptal edilir, yeniden başlanır. Dördüncü adımdaki Accounting Service transaction'ı kritiktir, pivottur. Kredi kartı onaylanırsa işlemin devamında sorun yoktur, işlem kesin olarak sonuçlanır. Onaylanmazsa geri dönüp tekrar denenmez, işlem tamamen iptal edilir. Sonraki onaylama adımları sorunsuz tamamlanır.

18. Karşı önlemler

Yalıtım eksikliğinin etkilerini azaltmak için farklı karşı önlem türleri vardır. Slayttaki liste (ekrandan, 00:50:00):

- **Semantic lock (anlamsal kilit):** Uygulama seviyesinde bir kilit.
- **Commutative updates (değişmeli güncellemeler):** Güncelleme işlemleri herhangi bir sırada çalıştırılabilecek şekilde tasarlanır.
- **Pessimistic view (karamsar görüş):** İş riskini en aza indirmek için saganın adımları yeniden sıralanır.
- **Reread value (değeri yeniden okuma):** Veri, üzerine yazılmadan önce yeniden okunup değişmediği doğrulanarak kirli yazmalar önlenir.
- **Version file (sürüm dosyası):** Bir kayda yapılan güncellemeler kaydedilir, böylece yeniden sıralanabilirler.
- **By value (değere göre):** Her isteğin iş riski kullanılarak eş zamanlılık mekanizması dinamik olarak seçilir.

Sunumdaki açıklamalar:

- **Semantic lock** (00:49:54-00:50:23): Saganın telafi edilebilir transaction'ının oluşturduğu ya da değiştirdiği kayda bir işaret (flag) konur. Bu işaret kaydın commit edilmediğini, henüz tamamlanmadığını ve değişebileceğini gösterir. İşaret, diğer transaction'ların kayda erişmesini engelleyen bir kilit ya da yalnız bir uyarı olabilir.
- **Commutative updates** (00:50:23): Güncelleme işlemi her sırada çalıştırılabilir hâle getirilir; bu, kayıp güncelleme sorununu önler.
- **Pessimistic view** (00:50:39): Kirli okumayı önlemek için saga adımları en karamsar durum düşünülerek, riski en aza indirecek şekilde yeniden düzenlenir.
- **Reread value** (00:50:55): Kayıp güncellemeleri önlemek için kullanılır. Bir kaydı güncellemeden önce yeniden okunur ve değişmediğinden emin olunur.
- **Version file** (00:51:06): Bir kayda yapılan güncellemeler kaydedilir; bu, onları yeniden sıralamaya imkân verir ve değişmeli olmayan (non-commutative) işlemleri değişmeli işlemlere dönüştürür.
- **By value** (00:51:27): Eş zamanlı işlemler arasında duruma göre seçim yapılır; her istek için saga mı yoksa dağıtık transaction mı kullanılacağına isteğin değerine (iş riskine) bakılarak karar verilir.

  Ek bilgi (derste anlatılmadı): Kitapta semantic lock'un örneği büyük olasılıkla siparişin `APPROVAL_PENDING` gibi bir `*_PENDING` durumuyla oluşturulmasıdır; bu durum siparişin bir saga tarafından güncellenmekte olduğunu diğer sagalara bildirir. Reread value ise büyük olasılıkla Optimistic Offline Lock kalıbının bir biçimi olarak sunulur: kayıt değişmişse saga durdurulur ve gerekirse yeniden başlatılır.

19. Bölüm 4 özeti ve hocanın değerlendirmesi

19.1 Sunumun özeti

Slayttaki özet (ekrandan, 00:51:50):

- Sistem işlemleri birden çok servise dağılmış veriyi güncellemek zorunda kalabilir.
- Geleneksel 2PC tabanlı dağıtık transaction'lar modern uygulamalar için uygun değildir.
- Bu yüzden Saga kalıbı kullanılır.
- Saga, mesajlaşmayla koordine edilen yerel transaction'lar dizisidir.

- Saganın adımlarını koordine etmek için koreografi ya da orkestrasyon kullanılabilir. Koreografi tabanlı sagada bir yerel transaction, diğer katılımcıların yerel transaction'larını çalıştırmasını tetikleyen olaylar yayımlar. Orkestrasyon tabanlı sagada merkezî bir saga orkestratörü, katılımcılara yerel transaction'larını çalıştırmalarını söyleyen komut mesajları gönderir.
- Saga tabanlı iş mantığı tasarlamak zor olabilir, çünkü ACID transaction'ların aksine sagalar birbirinden yalıtılmış değildir. Çoğu zaman karşı önlemler kullanmak gerekir: ACD transaction modelinin yol açtığı eş zamanlılık anomalilerini önleyen tasarım stratejileri.

19.2 Hocanın değerlendirmesi

Hoca sunumu şöyle bağlar (00:53:07-00:53:37): dağıtık, küçük servislerden oluşan bir yapıda bir sipariş (order) ya da satın alma (purchase) gibi daha büyük bir iş, yani bir senaryo nasıl yapılır, adımlar nasıl birleştirilir? Bunun bir yolu iki aşamalı commit'tir. Mademki dağıtık ve mikroservislerle çalışılıyor, servisler belli bir kalıba göre haberleşip bu işlemleri yapsın; buna saga kalıbı denir. İki seçenek vardır: koreografi ve orkestrasyon.

Koreografide tamamen dağıtıksınız; herkes kendi sorumluluğunu üstlenmiştir, ama yapıyı okumak, anlamak ve gerçeklemek daha zordur (00:53:37). Bunun üzerine sorumluluğu yeni bir uzmana, bir orkestra şefine verip her şeyi onun yönetmesini isteyebilirsiniz (00:54:30). Bu kez de sorumluluğu hangi sınıfa ya da hangi servise vereceğiniz, onun için ayrı bir servis açıp açmayacağınız sorunu çıkar. Bağlılık açısından daha iyi bir çözümdür (00:54:58).

> **Hocanın vurgusu:** Orkestratör ayrı bir hata noktası (point of failure) olarak karşınıza çıkabilir. Orkestra şefi arıza yaparsa ne olacak? Orkestra şefi tıkanırsa, ona çok fazla istek gelirse ne olacak? Bu yaklaşımın da kendine göre eksileri vardır (00:55:08-00:55:22).

Ne yapılırsa yapılsın, saga kalıbında yalıtım tam sağlanmadığı için transaction'larda sorunlar yaşanabilir (00:55:26). Hoca bunun günlük hayattan bir örneğini verir (00:55:44-00:56:27): Amazon'dan sipariş verirsiniz, sonra sipariş birden iptal edilir. Neden? Sagalar arada karışmıştır: stokta iki ürün varken, sagalar birbirinin ara durumunu okuduğu (kirli okuma) için dört sipariş alınmıştır. Şimdi iki siparişin iptal edilmesi gerekir; hangi ikisinin iptal edileceği de ayrı bir iş kararı mekanizmasıyla çözülür. Sonrasında hediye çeki ya da kupon tanımlanarak müşterinin gönlü alınmaya çalışılır.

> **Hocanın vurgusu:** Kritik işleriniz varsa, örneğin uzaya roket yolluyorsanız, saga yetmez; orada tamamen monolitik çalışmanız gerekebilir. İşin ucu hesaptaki paraların yazılmasına, muhasebeye gidiyorsa, mimarinizi istediğiniz kadar mikroservis yapın, bir yerde yine monolitik çalışmanız gerekecektir. Her derde mikroservis çözümü yoktur; uygulama uzayına hangi mimari daha uygunsa onu kullanmanız gerekir (00:56:27-00:57:16).

Örnek Sorular

Soru 1 (derste sözlü, 00:34:32): Bir e-ticaret sitesinde önce ödeme alınıyor, sonra ürün stoktan düşülüyor. Ödeme gerçekleştikten hemen sonra bir hata olursa ne yapılmalıdır? Ödeme ve stok farklı servislerde ve farklı veritabanlarındaysa bu nasıl çözülür?

> **Cevap (derste verildi):** Monolitte tek veritabanının transaction'ı ile geri alınır. Mikroservislerde iki yol anlatılır: 2PC ile dağıtık transaction (koordinatör hazırlık aşamasında herkesten "tamam" alırsa commit, biri olumsuzsa hepsini iptal eder) ya da saga. Sagada geri alma (rollback) yapılamaz, çünkü her yerel transaction commit edilmiştir; o ana kadar yapılan adımlar telafi edici transaction'larla sondan başa doğru geri alınır.

Soru 2 (derste sözlü, 00:55:08): Orkestrasyon tabanlı sagada bütün koordinasyonu bir orkestratöre vermenin riski nedir?

> **Cevap (derste verildi):** Orkestratör ayrı bir hata noktası olur: arıza yaparsa ya da çok fazla istek gelip tıkanırsa saga ilerleyemez. Sunumda ayrıca iş mantığının fazlasını orkestratörde merkezileştirme riski anılır. Buna karşılık bağlılık açısından koreografiden daha iyidir ve döngüsel bağımlılık oluşmaz.

Soru 3 (derste sözlü, 00:55:44): Bir alışveriş sitesinde verdiğiniz sipariş, onaylandıktan sonra neden iptal edilebilir?

> **Cevap (derste verildi):** Sagalar arasında yalıtım olmadığı için. Hocanın örneğinde stokta iki ürün varken sagalar birbirinin ara durumunu okuduğu (kirli okuma) için dört sipariş alınmıştır; sonradan iki siparişin iptal edilmesi gerekir ve hangilerinin iptal edileceğine ayrı bir iş kuralıyla karar verilir. Bu tür anomaliler karşı önlemlerle (semantic lock, pessimistic view, reread value vb.) azaltılır.

Dersin Toparlaması

Bu derste mikroservislerin nasıl haberleştiği ve birden çok servise yayılan işlemlerin nasıl tutarlı tutulduğu işlendi. Üçüncü bölümde etkileşim stilleri (bire bir/bire çok, senkron/asenkron), API tanımı ve anlamsal sürümleme, metin ve ikili mesaj biçimleri, senkron RPI kalıbı, REST ve olgunluk modeli, gRPC ve asenkron mesajlaşma görüldü. Hocanın özeti, RPC'nin telefon hattı gibi iki tarafı meşgul eden sıkı bağlı bir yapı olduğu, birbirini beklemeyen servislerin genişlemeye daha yatkın olduğu ve mesajların bir mesaj aracısı (kuyruk) üzerinden yönetilmesi gerektiğiydi.

Dördüncü bölümde, her servisin kendi veritabanı olduğu bir mimaride ACID transaction'ın neden zorlaştığı, 2PC'nin neden uygun olmadığı ve bunun yerine saga kalıbının nasıl kullanıldığı anlatıldı. Sagalar koreografi ya da orkestrasyonla koordine edilir, hata durumunda telafi edici transaction'larla geri alınır, ama yalıtım sağlamadıkları için (ACD) kayıp güncelleme ve kirli okuma gibi anomalilere açıktır; bunlar karşı önlemlerle azaltılır. Hocanın kapanış mesajı, her derde mikroservisin çözüm olmadığı, kritik işlerde monolitik çalışmanın gerekebileceğiydi. Sonraki derste kitabın 5. bölümü (mikroservis mimarisinde iş mantığı tasarımı) ve 6. bölümü (event sourcing) sunulacaktır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.